15.2.2 Read-through

Instead of managing both the database and the cache, the application layer interacts only with the cache system; the database management logic being hidden behind the caching API. Compared to the cache-aside use case, the data access logic is simplified since there is only one data source to communicate with.

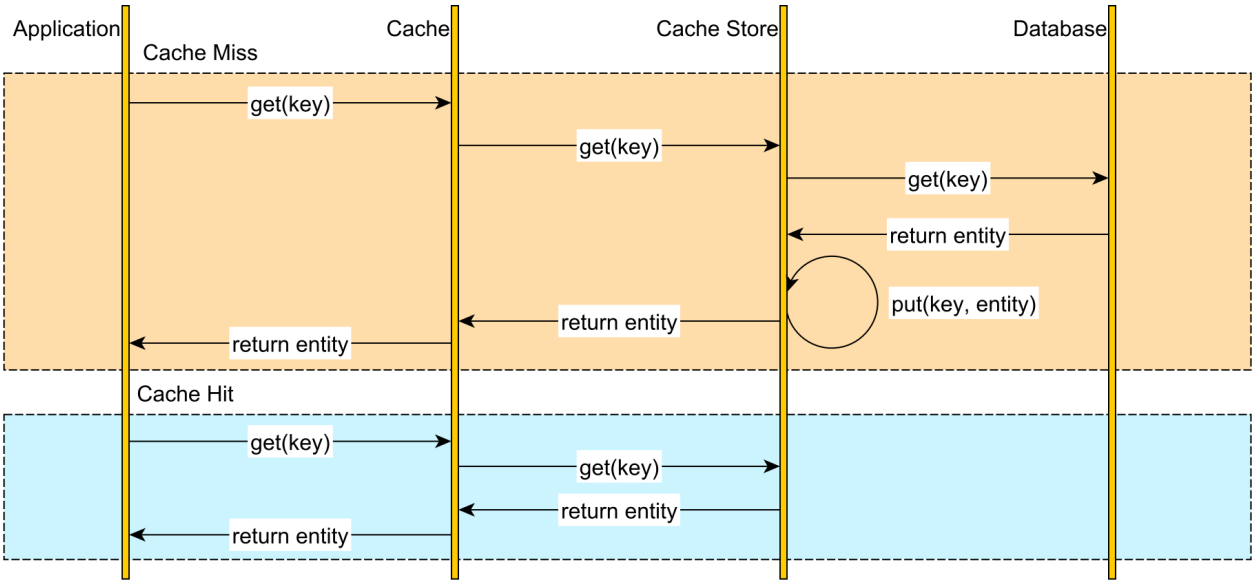

Figure 15.3: Read-through synchronization strategy

When fetching an entity, the cache checks if the requested entity is already contained in the cache store, and, upon a cache miss, the entity is loaded from the database.

15.2.3 Write-invalidate

If the entity is modified, the cache propagates the change to the underlying database and removes the associated entry from the cache. The next time this entity is requested, the cache system is going to load the latest version from the database.

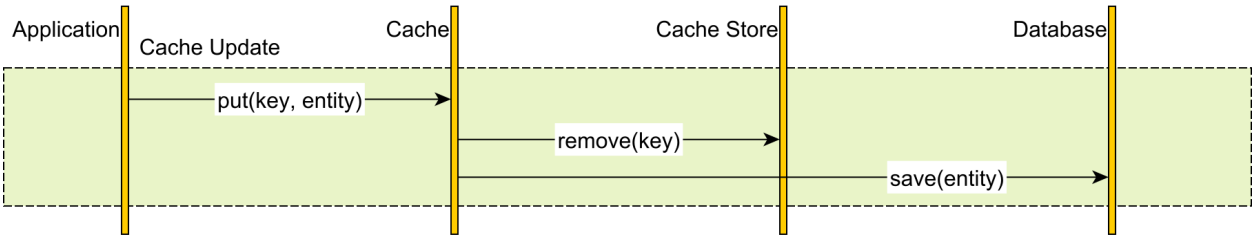

Figure 15.4: Write-invalidate synchronization strategy

15.2.4 Write-through

If the entity is modified, the changed is propagated to the underlying database and the cache as well.

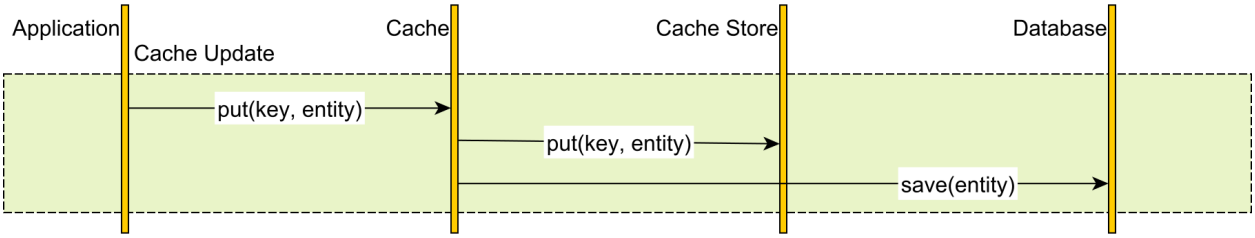

Figure 15.5: Write-through synchronization strategy

If the caching layer supports JTA transactions, the cache and the database can be committed at once. Although XA transactions can simplify development, the two-phase commit protocol incurs a significant performance overhead.

An alternative is to use soft locks on the cache side to hide the cache entry modification until the database transaction is committed, so that, until the lock is released, other concurrent transactions must load the entity from the database.

15.2.5 Write-behind

If strong consistency is not mandated, the change requests can be enqueued and flushed at once to the database.

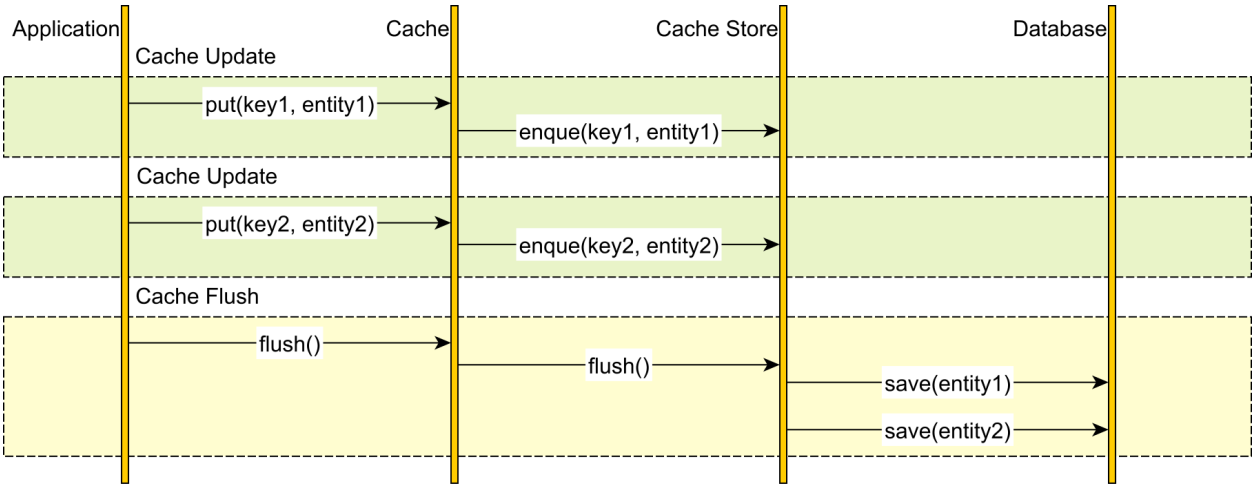

Figure 15.6: Write-behind synchronization strategy

This strategy is employed by the JPA Persistence Context, all entity state transitions being flushed towards the end of the currently running transaction or prior to executing a query.

15.3 Database caching

As explained at the beginning of this chapter, most database engines make use of internal caching mechanisms to speed up read and write operations. The most common database cache component is the in-memory buffers, but there might be other components as well such as the execution plan cache or query result buffer. Even without a database cache, the underlying operating system may offer caching for data pages.

Unlike application-level caches, the database cache does not compromise data consistency. Being both read-through and write-through, the database cache is transparent to the data access layer. Even with the advent of SSD (solid-state drive), disks still have a much higher latency than RAM. For this purpose, it makes much sense to load frequently-accessed data from memory, rather than going to disk.

Oracle

Oracle has multiple mechanisms for caching, such as:

- Buffer pool - storing blocks of data that are loaded from the underlying disk drive.
- Shared pool - storing parsed SQL statements, schema object metadata, sequence numbers.
- Large pool - stores results for parallel queries, large I/O buffers that are used for recovery management and backup or restore procedures.
- Result cache - stores results for SQL queries (when using the `RESULT_CACHE` query hint) and PL/SQL functions (when using the `RESULT_CACHE` directive).

On Unix systems, all I/O goes through the OS page cache. However, the same data is going to be cached in the Buffer pool, therefore data blocks are cached twice. For this reason, direct I/O[a] is desirable because it can bypass the file system cache, and the OS page cache can be used for other system processes.

There are also use cases when Oracle does not use the Buffer pool for caching data blocks (e.g. `TEMP` tablespace operations, `LOB` columns using the `NOCACHE` storing option), in which case the operating system cache may be suitable for speeding up read and write operations.

Although each caching structure can be configured manually, it is often a good idea to leave this responsibility to the automatic memory management[b] mechanism, which is enabled by default.

[a] http://docs.oracle.com/database/121/TGDBA/pfgrf_os.htm#TGDBA94410
[b] https://docs.oracle.com/database/121/TGDBA/memory.htm#TGDBA505

SQL Server

To provide very low transaction response times, SQL Server strives for reducing I/O operations (which are a source of performance-related issues in many database systems). For this reason, the database engine tries to use as much system memory as possible so that frequently-accessed data and index disk pages are served from RAM rather than the disk drive.

Upon startup, SQL Server allocates a portion of the system memory and uses it as a buffer pool. The buffer pool is divided into multiple pages of 8KB. Both data and index pages are read from disk into buffer pages, and, when the in-memory pages are modified, they are written back to disk.

SQL Server 2014 supports buffer pool extensions[a], which allow it to use SSD drives to increase the buffer cache size beyond the capabilities of the current system available memory.

[a] https://msdn.microsoft.com/en-us/library/dn133176.aspx

PostgreSQL

For improving read and write operation performance, PostgreSQL relies heavily on the underlying operating system caching capabilities. However, most operating systems use a LRU (least recently used) page replacement policy which is unaware of the data access patterns or other database-related considerations.

For this reason, PostgreSQL defines a shared buffers structure which stores disk pages into 8KB in-memory page cache entries. The shared buffer size is controlled via the `shared_buffers` configuration property. Unlike the OS cache, the shared buffers use a LFU (least frequently used) algorithm called clock sweep which counts the number of times a disk page is used. The more often a disk page is being used, the longer it is going to linger in the shared buffer database internal cache.

That being said, the shared buffer structure is more useful for storing frequently-accessed data blocks, while the operating system cache can be used for everything else. The shared buffer cache should not be set too high because the database engine requires memory for other operations as well (sorting, hashing, building indexes, vacuuming).

Although the shared buffers structure is very important for speeding up reads and writes, it is good practice to limit the shared buffer size[a] to the size of the current working set, therefore leaving enough memory for other database-related tasks.

[a] http://www.postgresql.org/docs/current/static/runtime-config-resource.html

MySQL

MySQL uses its internal buffer pool to cache data and indexes. The buffer pool is implemented as a linked list of memory pages. If the buffer pool size is smaller than the overall InnoDB tablespace size, a LRU-based algorithm is going to be used to deallocate older page entries.

The pool size is given by the `innodb_buffer_pool_size` configuration property which, ideally, should be adjusted so that it can hold all data and indexes in memory. Care must be taken to allow enough memory for the OS, as well for other MySQL structures and processes (e.g. threads allocated for each individual connection, sort buffers, query cache).

On Linux, to avoid double buffering caused by the operating system caching mechanism, the `innodb_flush_method`[a] configuration property should be set to `O_DIRECT`.

Nevertheless, the OS cache is useful for storing the InnoDB transaction log (used for ensuring ACID transactions), the binary log (used for database replication), and other MySQL structures that are not covered by the InnoDB buffer pool.

[a] http://dev.mysql.com/doc/refman/5.7/en/innodb-parameters.html#sysvar_innodb_flush_method

Essential, but not sufficient

Database caching is very important for a high-performance enterprise application. However, database caching only applies to a single node, and, if the database size is bigger than the capacity of a single node, then this solution alone is no longer sufficient. One workaround is to use database sharding, but that is not without challenges.

Even if database caching improves performance considerably, the networking overhead still plays a significant role in the overall transaction response time. If the application operates on graphs of entities, fetching an entire graph might require lots of joins or many secondary select statements. For this reason, it makes sense to cache the whole entity aggregate and have it closer to the application layer.

If the enterprise system relies only on the database system alone to serve read requests, the database becomes a single point of failure. This availability can be increased by using database replication. However, if all database nodes are collocated (to reduce the synchronization overhead caused by networking latency), the database system can still become unavailable if the data center is facing a sudden power outage.

For all these reasons, it is good practice to use an application-layer caching solution to address database caching limitations.

15.4 Application-level caching

Application caches are a necessity for high-performance enterprise applications, and this section is going to explore this topic in greater detail. No matter how well tuned a database engine is, the statement response time is highly dependent on the incoming database load. A traffic spike incurs a high contention of database system resources, which can lead to higher response times.

For instance, most internet applications expose a sitemap which is used by Search Engine bots to index the content of the site in question and make it available for searches. From a business perspective, having a high page rank is highly desirable because it can translate to more revenue. However, the Search Engine bot can generate a sudden traffic spike, which, in turn, can lead to a spike in transaction response time. Unfortunately, high response times can affect the site page rank. That being said, the translation response time must be relatively low even during high traffic loads.

The application-level cache can, therefore, level up traffic spikes because the cache fetching complexity is O(1). More, if the application-level cache holds a significant portion of the entire data set, the application can still work (even if in a read-only mode) when the database is shut down for maintenance or due to a catastrophic event.

15.4.1 Entity aggregates

In a relation database, data is normalized, and, for a complex enterprise application, it is usually spread across multiple tables. On the other hand, the business logic might operate on entity graphs which assemble information from various database tables.

To better visualize the entity aggregate, consider the following diagram depicting all entities associated with a single `Post` in a particular forum.

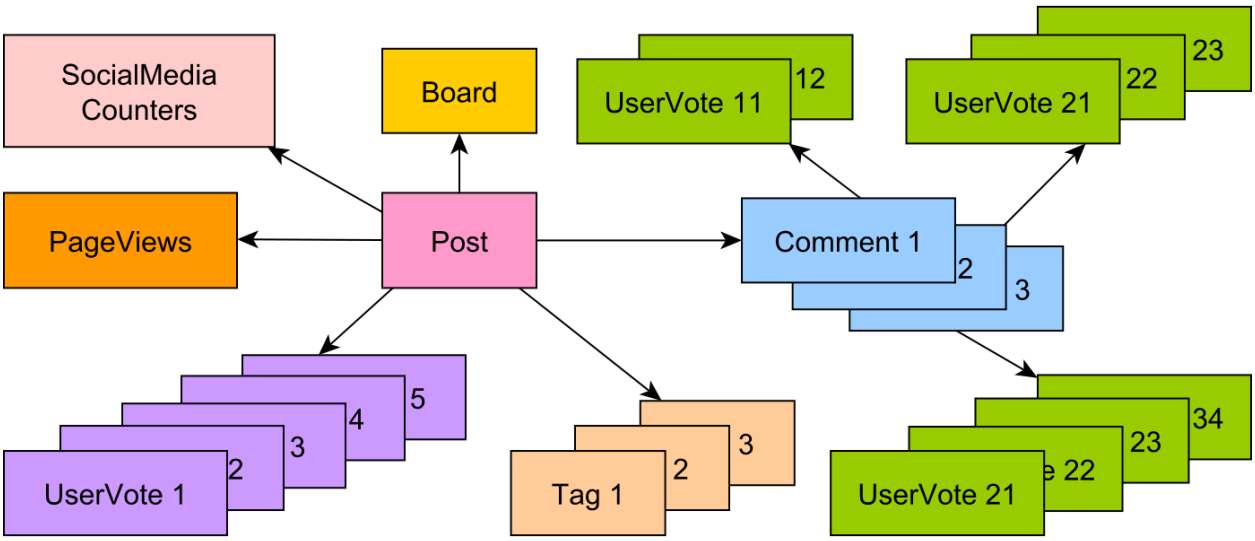

Figure 15.7: Entity aggregates

The `Post` entity is the root since all other entities relate to it, either directly or indirectly. A `Post` belongs to a `Board` entity, and it can have several `Tag(s)`. The `PageViews` entity summarizes statistics about how popular a given `Post` might be. There is also a `SocialMediaCounters` entity to hold the number of shares for social media platforms. Users can add `Comment(s)` to a `Post`, and they can also cast a vote on both the `Post` or the `Comment` entity.

The sitemap contains the list of all `Post(s)` so that Search Engines can index all questions and answers. When a `Post` is requested, the whole entity aggregate is required to render the display. Without application-level caching, the data access layer would have to either join all the associated entities or use secondary select statements.

To avoid a Cartesian Product, the `Post` entity should be joined to its `Tag(s)`, as well as with other `many-to-one` relationships (e.g. `Board`, `PageViews`, `SocialMediaCounters`). A secondary query is used to fetch the `UserVote(s)` associated with the current `Post`. The `Comment(s)` can be fetched with a secondary select, and this is desirable since there might be many `Comment(s)`, and the secondary query can better use pagination. The `Comment` query could also join the `UserVote(s)` so that these two entities are fetched with a single query as well.

15.4.2 Distributed key-value stores

While the underlying data resides in the relational database, the entity aggregate can also be saved in a distributed cache, such as Redis or Memcached. Key-value stores are optimized for storing data structures in memory, and the lookup complexity is O(1). This is ideal for high-performance enterprise applications since response time can stay low even during unforeseen traffic spikes.

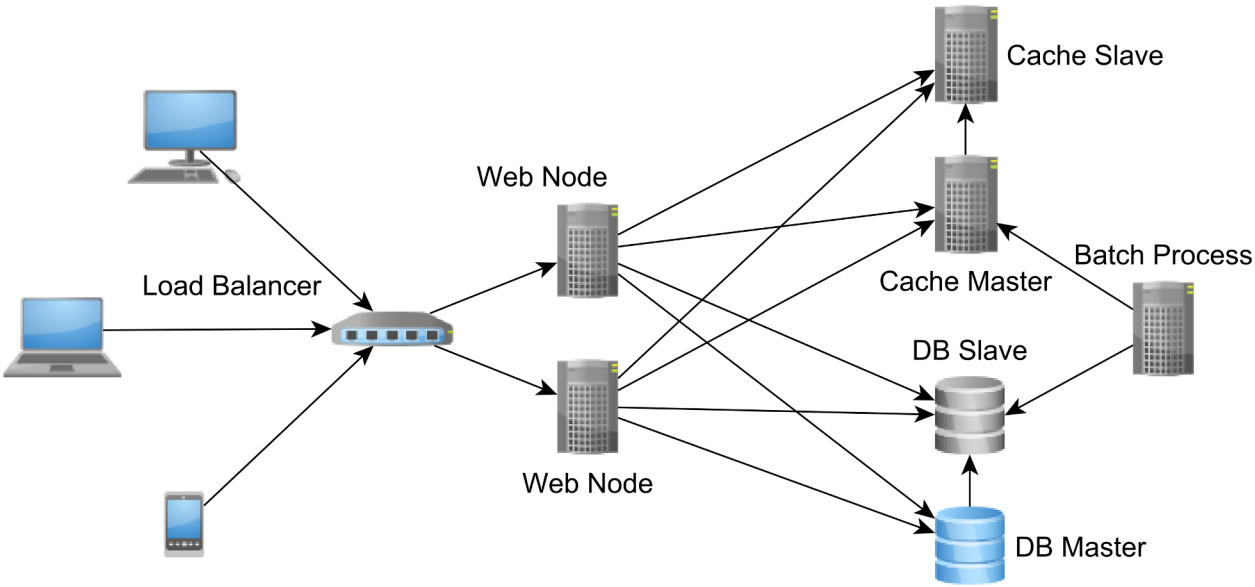

Figure 15.8: Application-level cache integration

The relational database is still the system of record, while the key-value caching solution is used as an alternate data provider.

15.4.3 Cache synchronization patterns

Unfortunately, duplicating data among two data sources is not without issues. Ideally, both the relational database and the application-level cache should be in sync, every update being applied synchronously to both data stores. In reality, not all business use cases have strict consistency requirements.

For instance, the `PageViews` and the `SocialMediaCounters` can be updated periodically by a batch processor which can aggregate the database counters and update the cache entry with the latest aggregated values. On the other hand, some actions need a more strict consistency guarantees. For `Comment` entries, read-your-writes consistency is needed because otherwise users might miss their own changes.

Caching is always about trade-offs, and not all business use cases are equal in terms of consistency guarantees. For strict consistency, some transactions might need to bypass the cache entirely and read from the database.

Once a user or a batch process makes a change to the underlying database records, the cache needs to be updated as well. As explained in the cache concurrency strategies section, there are several ways to implement the synchronization logic.

15.4.4 Synchronous updates

If cache-aside is being used, the business logic must update both the database and all associated cache entries in the same transaction. Because most key-value stores do not use XA transactions, the cache entries can be either invalidated (in which case there is no risk of reading stale data), or they can be updated after the database transaction has been committed (in which case there is a slight time interval when a concurrent transaction can read a stale entry from the cache).

For the previous Domain Model, `Comment` entities should be processed synchronously. Adding or modifying a `UserVote` entry can also be done synchronously, or at least for the comments that are associated with the currently logged user.

15.4.5 Asynchronous updates

If eventual consistency is tolerated, then asynchronous updates are also a viable solution, and the application logic can be simplified since the caching logic is decoupled from business logic. This is also necessary when there are multiple data stores that need to be updated according to the latest changes that happened in the database. For instance, the enterprise

application might need to propagate changes to a cache, an in-memory data processing framework (e.g. Spark) which might monitor the forum for spam messages, or to a data warehouse. In this case, the changes must be captured from the database and propagated to all other subsystems that are interested in being notified about these updates.

15.4.5.1 Change data capture

In database terminology, change data capture (CDC) is an assembly of patterns that are responsible for recording database changes.

One solution is to record the timestamp version of every row that is either inserted, updated, or deleted. This pattern only works if records are not actually physically removed, but instead, they are simply marked as deleted (soft deleting), and hidden away from any database query.

Another implementation would be to capture changes using database triggers so that an event is recorded whenever a row is inserted, updated, or deleted. Unfortunately, triggers might slow down write operations which is undesirable especially if there is only one database Master node because the longer the write transactions take, the less throughput the Master node will accommodate.

A more efficient approach is to use a framework that can parse the database transaction log. Unlike database triggers, this approach does not incur any additional performance penalty for write operations since the transaction log is being parsed asynchronously. The only drawback is that not all database support this natively, and the transaction log entries can change from one database version to the other.

Oracle

Oracle GoldenGate[a] is a change data capture tool that can be used either for database replication or as an ETL (extract, transform, and load) process in order to feed a data warehouse. Another approach is to use Databus[b], which is an open-source framework developed by Linkedin for log mining.

[a] http://www.oracle.com/us/products/middleware/data-integration/goldengate/overview/index.html
[b] https://github.com/linkedin/databus

SQL Server

Since version 2008, SQL Server offers a Change Data Capture[a] solution that can be configured at the database, table, or even column level.

[a] https://msdn.microsoft.com/en-us/library/cc627369.aspx

PostgreSQL

Even if there is no native CDC solution, PostgreSQL 9.4 has introduced logical decoding[a] which can be used for extracting row-level modifications.

[a] http://www.postgresql.org/docs/current/static/logicaldecoding.html

MySQL

There are multiple solutions that are able to parse the MySQL binary log, and the most notable is Databus which supports both Oracle and MySQL.

Denormalization ripple effect

In the previous Domain Model, storing the `Board` and the list of `Tag(s)` associated with every particular `Post` entity graph is appropriate only if the `Board` and the `Tag` are practically immutable. Otherwise, changing the `Board` entity could ripple throughout the cache, causing a large number of entries to be updated as well. This problem is even more acute if cache entry invalidation is being used. For `Tag(s)`, the `Post` aggregate should store only a list of `Tag` identifiers, the actual `Tag` names being resolved upon fetching the `Post` entity aggregate from the cache.

On the other hand, `Comment(s)` and `UserVote(s)` are more related to a single `Post` entry, so they are more suitable for being stored in the `Post` entity aggregate. To avoid the ripple effect, the `UserVote` entity should only contain virtually immutable user-related columns (e.g. user identifier).

The higher the data denormalization degree associated with entity aggregates, the bigger the data change ripple. Therefore, it is good practice to avoid storing associations that might be shared among many entity graphs cache entries.

15.5 Second-level caching

While the Persistence Context has long been referred to as the first-level cache, in reality, it is meant to provide application-level repeatable reads rather than lowering fetch execution time. The first-level cache is not thread-safe, and, once the Hibernate `Session` is closed, the cached entities are no longer accessible.

On the other hand, the second-level cache is bound to a `SessionFactory`, it is thread-safe, and it provides a solution for optimizing entity aggregate loading time. Hibernate only defines the contract for the second-level cache API and does not provide a reference implementation for this specification. The second-level cache API is implemented by third-party caching providers, such as Infinispan[1], Ehcache[2], or Hazelcast[3].

Being tightly integrated with Hibernate, the second-level cache does not require any data access layer code change. While application-level caches operate in a cache-aside synchronization mode, the second-level cache offers read-through and write-through cache update strategies.

Unlike an application-level caching solution, the second-level does not store entity aggregates. Instead, entities are saved in a row-level data format which is closer to the associated database row values. Although it features a collection-level cache component, behind the scenes, it only saves the entity identifiers contained in a particular collection instance. The same is true for the entity query caching, whose cache entries contain only the entity identifiers that satisfy a given query filtering criteria.

For all the aforementioned reasons, the second-level cache is not a replacement or a substitute for application-level caches. The biggest gain for using the Hibernate second-level cache is that, in a Master-Slave database replication scheme, it can optimize read-write transactions. While read-only queries can be executed on many Slave nodes, read-write transactions can only be executed by the Master node.

Being capable of working in read-through and write-through mode, the second-level cache can help reduce read-write transactions response time by reducing the amount of work the Master node is required to do.

[1] http://infinispan.org/
[2] http://www.ehcache.org/
[3] http://hazelcast.org/

15.5.1 Enabling the second-level cache

By default, the `hibernate.cache.use_second_level_cache` configuration is set to `true`. However, this is not sufficient because Hibernate requires a `CachingRegionFactory` implementation as well, and, without specifying any third-party implementation, Hibernate defaults to using the `NoCachingRegionFactory` implementation, meaning that nothing is actually being cached.

For this reason, it is mandatory to supply the `hibernate.cache.region.factory_class` configuration property, which takes the fully-qualified class name of the `CacheRegionFactory` third-party implementation.

```
<property name="hibernate.cache.region.factory_class"
          value="org.hibernate.cache.ehcache.EhCacheRegionFactory"/>
```

After enabling the second-level cache, the application developer must instruct Hibernate which entities should be cached. Although JPA 2.0 defined the `@Cacheable` annotation, Hibernate also requires a cache concurrency strategy.

For this reason, the `org.hibernate.annotations.Cache` annotation should be provided as well.

```
@Entity
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Post {

    //Fields, getters, and setters omitted for brevity
}
```

Hibernate defines the `hibernate.cache.default_cache_concurrency_strategy` configuration property which applies the same synchronization strategy to all cacheable entities. When this configuration property is set, the `@Cache` annotation is no longer mandatory, and the `@Cacheable` annotation can be used instead. By supplying a `@Cache` annotation, the default cache concurrency strategy can be overridden on a per-entity basis.

15.5.2 Entity cache loading flow

Once the second-level cache is activated for a particular entity, it participates automatically in the entity loading mechanism.

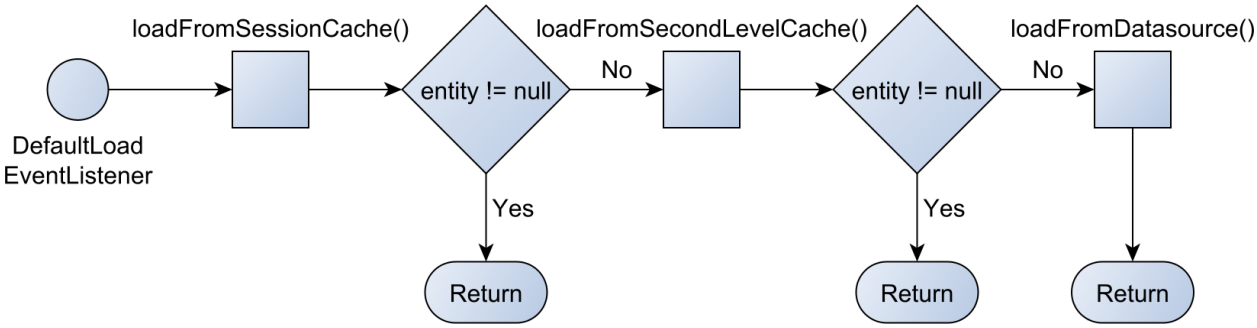

Figure 15.9: Entity loading control flow

When loading an entity, Hibernate always checks the Persistence Context first. This behavior guarantees application-level repeatable reads. Once an entity becomes managed, Hibernate will use the same entity instance when loading it directly or including it in an entity query.

If the entity is not found in the currently running Persistence Context and the second-level cache is configured properly, Hibernate checks the second-level cache. Only if the second-level cache does not contain the entity in question, Hibernate will fetch the entity from the underlying database.

15.5.3 Entity cache entry

Internally, every entity is stored as a `CacheEntry`. As previously explained, Hibernate does not store aggregates, and the second-level cache entry is close to the underlying table row representation.

Hydrated and disassembled state

In Hibernate nomenclature, *hydration* represents the process of transforming a JDBC `ResultSet` into an array of raw values.

The *hydrated state* is saved in the currently running Persistence Context as an `EntityEntry` object which encapsulated the loading time entity snapshot. The hydrated state is then used by the default dirty checking mechanism which compares the current entity data against the loading time snapshot.

The second-level cache entry values contain the hydrated state of a particular entity. However, for the second-level cache the hydrated state is called *disassembled state*.

To visualize the disassembled entity state, consider the following entity model:

```
@Entity @Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Post {
    @Id
    private Long id;

    private String title;

    @Version
    private int version;

    //Getters and setters omitted for brevity
}

@Entity @Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class PostDetails {
    @Id
    private Long id;

    private Date createdOn;

    private String createdBy;

    @OneToOne
    @MapsId
    private Post post;

    //Getters and setters omitted for brevity
}

@Entity @Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class PostComment {
    @Id
    private Long id;

    @ManyToOne
    private Post post;

    private String review;

    //Getters and setters omitted for brevity
}
```

Upon saving and fetching the following `Post` entity:

```
Post post = new Post();
post.setId(1L);
post.setTitle("High-Performance Java Persistence");
entityManager.persist(post);
```

Hibernate stores the following second-level cache entry:

```
item = {org.hibernate.cache.ehcache.internal.strategy.AbstractReadWriteEhcacheAccess\
Strategy$Item}
  value = {org.hibernate.cache.spi.entry.StandardCacheEntryImpl}
    disassembledState = {java.io.Serializable[1]}
      0 = "High-Performance Java Persistence"
    subclass = "com.vladmihalcea.book.hpjp.hibernate.cache.Post"
  version = 0
  timestamp = 5990528746983424
```

The `disassembledState` is an `Object[]` array which, in this case, contains a single entry that represents the `Post` title. The `version` attribute is stored separately, outside of the `disassembledState` array. The entity identifier is stored in the cache entry key which looks as follows:

```
key = {org.hibernate.cache.internal.OldCacheKeyImplementation}
  id = {java.lang.Long} "1"
  type = {org.hibernate.type.LongType}
  entityOrRoleName = "com.vladmihalcea.book.hpjp.hibernate.cache.Post"
  tenantId = null
  hashCode = 31
```

The cache entry key contains the entity type (e.g. `entityOrRoleName`), the identifier (e.g. `id`), and the identifier type (e.g. `type`). When multitenancy is being used, the tenant identifier (e.g. `tenantId`) is stored as well.

When storing a `PostDetails` entity:

```
PostDetails details = new PostDetails();
details.setCreatedBy("Vlad Mihalcea");
details.setCreatedOn(new Date());
details.setPost(post);
entityManager.persist(details);
```

The second-level cache entry looks like this:

```
item = {org.hibernate.cache.ehcache.internal.strategy.AbstractReadWriteEhcacheAccess\
Strategy$Item}
  value = {org.hibernate.cache.spi.entry.StandardCacheEntryImpl}
    disassembledState = {java.io.Serializable[3]}
      0 = "Vlad Mihalcea"
      1 = {java.util.Date} "Fri May 06 15:45:10 EEST 2016"
    subclass = "com.vladmihalcea.book.hpjp.hibernate.cache.PostDetails"
  version = null
  timestamp = 5990558557458432
```

The `version` attribute is null because the `PostDetails` entity does not feature a `@Version` attribute. The `disassembledState` array has a length of 3, although just the `createdBy` and the `createdOn` attributes are visible. The `@OneToOne` association information is stored as null in the `disassembledState` array because Hibernate knows that the entity identifier is sufficient to locate the associated parent relationship.

When persisting a `PostComment` entity:

```
PostComment comment1 = new PostComment();
comment1.setId(1L);
comment1.setReview("JDBC part review");
comment1.setPost(post);
entityManager.persist(comment1);
```

The disassembled state will contain the `review` attribute and the foreign key value that is used for identifying the `@ManyToOne` association:

```
item = {org.hibernate.cache.ehcache.internal.strategy.AbstractReadWriteEhcacheAccess\
Strategy$Item}
  value = {org.hibernate.cache.spi.entry.StandardCacheEntryImpl}
    disassembledState = {java.io.Serializable[2]}
      0 = {java.lang.Long} "1"
      1 = "JDBC part review"
    subclass = "com.vladmihalcea.book.hpjp.hibernate.cache.PostComment"
  version = null
  timestamp = 5990563491569665
```

15.5.3.1 Entity reference cache store

Hibernate can also store entity references directly in the second-level cache, therefore avoiding the performance penalty of reconstructing an entity from its disassembled state. However, not all entity types are allowed to benefit from this optimization.

For an entity to be cached as a reference, it must obey the following rules:

- The entity must be immutable, meaning that it must be marked with the `@org.hibernate.annotations.Immutable` annotation.
- It might not feature any entity association (`@ManyToOne`, `@OneToOne`, `@OneToMany`, `@ManyToMany`, or `@ElementCollection`).
- The `hibernate.cache.use_reference_entries` configuration property must be enabled.

Among the previously Domain Model entities, only the `Post` entity could be stored as an entity reference because `PostDetails` has a `@OneToOne Post` association, while `PostComment` has a `@ManyToOne Post` relationship. Therefore, the `Post` entity only needs to be marked with the `@Immutable` annotation:

```
@Entity @Immutable
@Cache(usage = CacheConcurrencyStrategy.READ_ONLY)
public class Post implements Serializable {

    @Id
    private Long id;

    private String title;

    @Version
    private int version;

    //Getters and setters omitted for brevity
}
```

It is good practice to make the entity `Serializable` because the cache provider might need to persist the entity reference on disk. Because entities are immutable, the `READ_ONLY` is the most obvious `CacheConcurrencyStrategy` to use in this case.

When storing the same `Post` entity instance that was used for the disassembled state use case, the cache entry value is going to look as follows:

```
value = {org.hibernate.cache.spi.entry.ReferenceCacheEntryImpl}
  reference = {com.vladmihalcea.book.hpjp.hibernate.cache.Post}
    id = {java.lang.Long} "0"
    title = "High-Performance Java Persistence"
    version = 0
  subclassPersister = {org.hibernate.persister.entity.SingleTableEntityPersister}
```

To understand the performance gain for storing and retrieving entity references, the following test case is going to measure how much time it takes to fetch 100, 500, 1000, 5000, and 10 000 entities from the second-level cache when using the default entity disassembled state mechanism or the entity reference cache store.

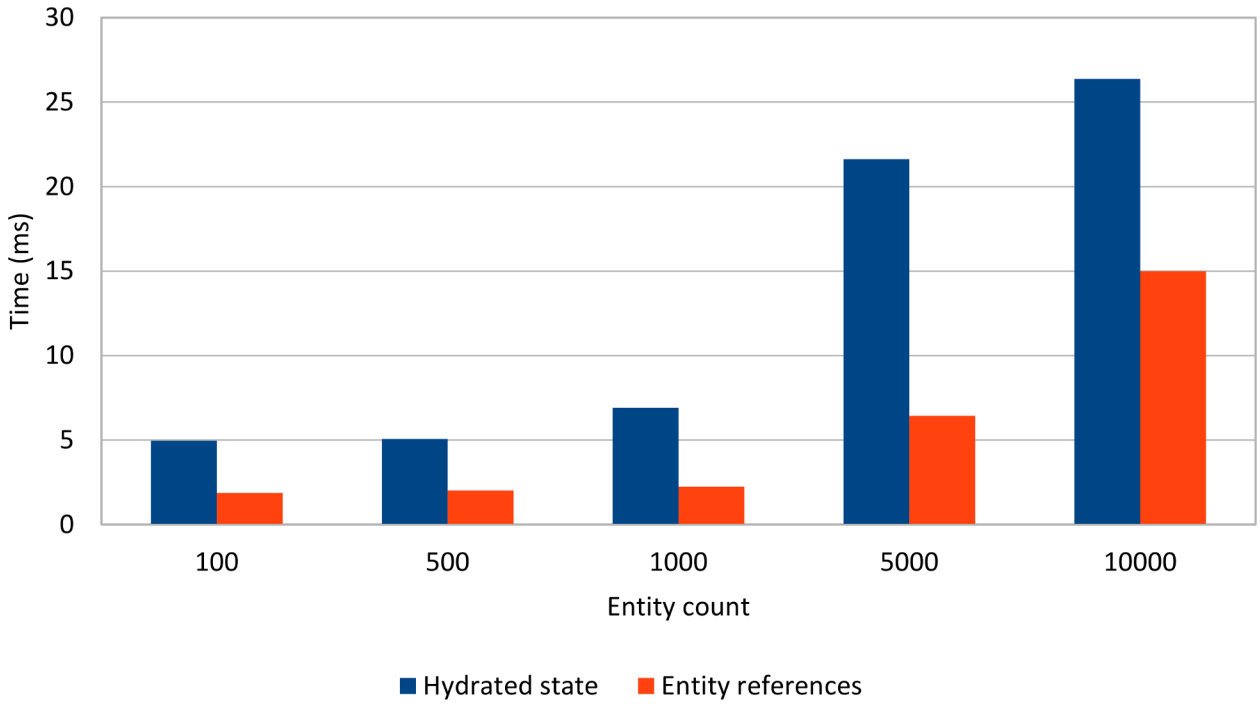

Figure 15.10: Disassembled state vs Entity references

Fetching entity references is much more efficient since new objects are not required to be instantiated and populated with the entity disassembled state. The more entities are fetched from the cache, the more apparent the time gap between the default entity cache store and its entity reference alternative.

Although the `hibernate.cache.use_reference_entries` configuration allows reducing the cache fetching time, it's not a general purpose second-level cache optimization technique because it's only applicable to entities that do not have any association mapping.

15.5.4 Collection cache entry

The collection cache allows storing the entity identifiers that are contained within a given collection instance. Because it only stores identifiers, it is mandatory that the contained entities are cached as well.

The collection cache is activated by the `hibernate.cache.use_second_level_cache` configuration property, just like the regular entity caching mechanism.

The `Post` entity has a bidirectional one-to-many `PostComment` association that is mapped as follows:

```
@OneToMany(cascade = CascadeType.ALL, mappedBy = "post", orphanRemoval = true)
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
private List<PostComment> comments = new ArrayList<>();
```

For the next example, two `PostComment` child entities are going to be associated with a managed `Post` entity:

```
Post post = entityManager.find(Post.class, 1L);

PostComment comment1 = new PostComment();
comment1.setId(1L);
comment1.setReview("JDBC part review");
post.addComment(comment1);

PostComment comment2 = new PostComment();
comment2.setId(2L);
comment2.setReview("Hibernate part review");
post.addComment(comment2);
```

Because the collection is marked with the `@Cache` annotation, upon accessing the collection for the first time, Hibernate is going to cache its content using the following cache entry key:

```
key = {org.hibernate.cache.internal.OldCacheKeyImplementation}
  id = {java.lang.Long} "1"
  type = {org.hibernate.type.LongType}
  entityOrRoleName = "com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments"
  tenantId = null
  hashCode = 31
```

The collection cache entry key is almost identical with the entity cache one, the only difference being the cache region name, which is constructed by appending the collection attribute name to the fully-qualified entity class name.

As previously explained, the associated cache entry value contains `PostComment` entity identifiers that are contained within the currently used `Post` entity `comment` collection:

```
item = {org.hibernate.cache.spi.entry.CollectionCacheEntry}
  state = {java.io.Serializable[2]}
    0 = {java.lang.Long} "1"
    1 = {java.lang.Long} "2"
```

Along with the entity cache, the collection cache allows retrieving an entity aggregate without having to hit the database even once. Although fetching an entire entity graph requires multiple cache calls, the major advantage of storing entities and collections separately is that invalidation or updates affect a single cache entry.

15.5.5 Query cache entry

Just like the collection cache, the query cache is strictly related to entities, and it draws an association between a search criteria and the entities satisfying the given filtering condition. The query cache is disabled by default, and, to activate it, the following configuration property needs to be supplied:

```
<property name="hibernate.cache.use_query_cache" value="true"/>
```

Even if the query cache is enabled, queries must be explicitly marked as cacheable. When using the Hibernate native API, the `setCacheable` method must be used:

```
List<Post> posts = (List<Post>) session.createQuery(
        "select p from Post p " +
        "where p.title like :token")
.setParameter("token", "High-Performance%")
.setCacheable(true)
.list();
```